

# INSTITUTO SUPERIOR UNIVERSITARIO JAPÓN

## CARRERA DE DESARROLLO DE SOFTWARE

ASIGNATURA: DESARROLLO DE APLICACIONES MÓVILES

### INFORME DE INVESTIGACIÓN Y DESARROLLO TÉCNICO

## DISPOSITIVOS REALES Y VIRTUALES, CONSUMO DE SERVICIOS WEB REST MEDIANTE AXIOS Y PROCESO DE GENERACIÓN DE APK EN APLICACIONES MÓVILES

#### DATOS INFORMATIVOS:

**DOCENTE / TUTOR:** Ing. Luis Calo  
**ESTUDIANTE / AUTOR:** Estudiante de Desarrollo de Software  
**TEMA ASIGNADO:** Dispositivos, APIs REST con Axios y APKs  
**CALIFICACIÓN:** Rúbrica de Recuperación (10 Puntos)  
**PERÍODO ACADÉMICO:** 2026 | Quito - Ecuador

## 1. TEMA Y OBJETIVOS DE LA INVESTIGACIÓN

### 1.1. Delimitación del Tema

El presente trabajo de investigación aborda tres pilares fundamentales en la ingeniería de software para dispositivos móviles: la evaluación y contraste entre entornos de ejecución físicos y virtuales (Emuladores, Simuladores y Android Virtual Device - AVD), la arquitectura de comunicación basada en servicios web REST sobre el protocolo HTTP/HTTPS utilizando el cliente avanzado Axios, y el ciclo de compilación, optimización de memoria (zipalign) y firmado criptográfico (apksigner) para la generación de archivos ejecutables APK en la plataforma Android.

### 1.2. Objetivo General

Analizar, contrastar e implementar los fundamentos teóricos y prácticos relativos a los entornos de prueba móviles, el consumo de servicios web REST mediante la biblioteca Axios y el ciclo completo de empaquetado y firmado de paquetes de aplicación APK en Android, desarrollando una solución práctica modular para evidenciar el comportamiento de cada alternativa tecnológica.

### 1.3. Objetivos Específicos

- Investigar y categorizar los principios de funcionamiento, arquitectura de bajo nivel, ventajas y desventajas de dispositivos móviles reales, emuladores basados en hipervisores (QEMU/AVD) y simuladores de software.
- Examinar la arquitectura RESTful y el protocolo HTTP/HTTPS en redes móviles, analizando las capacidades técnicas de Axios frente a alternativas nativas como Fetch en términos de interceptores, gestión de timeouts y control de excepciones.
- Describir el pipeline de compilación de Android (bytecode DEX, compilador D8/R8, empaquetador AAPT2, Gradle) y el procedimiento de firmado digital con keytool, zipalign y apksigner bajo esquemas v1, v2 y v3.
- Construir una aplicación móvil funcional que integre un cliente Axios centralizado con operaciones CRUD en tiempo real y documentar los scripts de despliegue y construcción de APK para su validación en entornos reales y virtuales.

## 2. MARCO TEÓRICO

### 2.1. Dispositivos Reales y Dispositivos Virtuales

El desarrollo de aplicaciones móviles exige verificar la correcta ejecución del software en múltiples dimensiones de hardware, arquitecturas de procesamiento y versiones del sistema operativo. La fragmentación del ecosistema Android requiere una estrategia integral de pruebas que combine dispositivos físicos y plataformas virtualizadas.

#### 2.1.1. Dispositivo Móvil Real (Hardware Nativo)

¿Qué es? Un dispositivo móvil real es una unidad física comercial (smartphone, tablet o dispositivo embebido) equipada con un circuito integrado System-on-Chip (SoC) basado en arquitectura ARM o ARM64, memoria RAM, módulos de almacenamiento no volátil, circuitos de radiofrecuencia (4G/5G, Wi-Fi, Bluetooth Low Energy, NFC) y un conjunto de sensores físicos (GPS, acelerómetro, giroscopio, cámaras y lectores biométricos).

¿Cómo funciona y su Arquitectura? La aplicación se ejecuta de forma nativa sobre el Android Runtime (ART) con compilación híbrida AOT (Ahead-of-Time) y JIT (Just-in-Time). El runtime interactúa directamente con el kernel de Linux y los controladores de la capa de abstracción de hardware (HAL). La comunicación de depuración con el equipo de desarrollo se establece a través del protocolo Android Debug Bridge (ADB), ya sea por interfaz física USB o conexión TCP/IP inalámbrica.

Ventajas: Ofrece una fidelidad absoluta (100%) en la respuesta táctil, tasa de refresco (Hz) y gestos multi-touch. Permite medir con exactitud métricas críticas de ingeniería como el consumo energético de la batería en miliamperios (mAh), el estrangulamiento térmico (thermal throttling) ante cargas prolongadas de CPU/GPU y el comportamiento ante caídas de señal de red móvil.

Desventajas: Representa un costo económico elevado para adquirir múltiples modelos de diferentes fabricantes (Samsung, Xiaomi, Motorola, Google). Sufre degradación física de baterías por ciclos continuos de carga durante el desarrollo y presenta dificultades para escalar en pipelines de integración continua (CI/CD).

#### 2.1.2. Emuladores vs. Simuladores: Diferencias de Arquitectura

En el ámbito de la virtualización móvil, existe una distinción técnica fundamental entre emuladores y simuladores:

- **Emulador (Hardware + Software):** Modela tanto la arquitectura de hardware como la pila de software del dispositivo objetivo. Emula las instrucciones de la CPU, registros, memoria y controladores de periféricos mediante hipervisores y traducción binaria (Binary Translation). Un ejemplo es el Android Virtual Device (AVD), el cual ejecuta el kernel Linux real de Android y todo el sistema operativo sobre QEMU y aceleradores de hardware como KVM en Linux o HAXM/Hyper-V en Windows.
- **Simulador (Solo Software):** Modela únicamente la interfaz de usuario, las bibliotecas y las APIs de alto nivel del sistema operativo objetivo, ejecutándose directamente sobre el hardware y el kernel del equipo anfitrión. Un ejemplo es el iOS Simulator de Xcode sobre macOS o los simuladores de viewport web en navegadores. No emula hardware de bajo nivel ni traduce instrucciones de procesador.

### 2.1.3. Android Virtual Device (AVD)

¿Qué es y cómo funciona? Un AVD es una configuración de emulación administrada por el Android SDK que combina un perfil de hardware (tamaño de pantalla, densidad DPI, memoria RAM) con una imagen del sistema (System Image x86\_64 o ARM64). Funciona sobre el motor QEMU 2.0+ utilizando virtualización asistida por hardware y aceleración gráfica por GPU (OpenGL/Vulkan passthrough).

**Ventajas del AVD:** Permite probar aplicaciones en diversas versiones de Android (desde API 21 hasta API 35+) sin adquirir hardware adicional. Facilita la simulación de eventos externos como llamadas telefónicas entrantes, envío de SMS, variación de nivel de batería, rotación de pantalla y rutas GPS personalizadas. Es fundamental para pruebas automatizadas con frameworks como Espresso o Appium en servidores CI/CD.

**Desventajas del AVD:** Requiere considerables recursos de hardware en la computadora anfitriona (mínimo 16 GB de RAM y procesador con soporte de virtualización VT-x/AMD-V). No reproduce con exactitud el comportamiento térmico real ni las peculiaridades de capas de personalización propietarias de los fabricantes (como One UI o MIUI).

**Tabla 1. Matriz Técnica Comparativa de Entornos de Ejecución Móvil**

*Nota: Tabla comparativa estructurada bajo parámetros técnicos de ingeniería.*

| Criterio Técnico        | Dispositivo Real         | Emulador AVD (QEMU)           | Simulador (Software)      |
|-------------------------|--------------------------|-------------------------------|---------------------------|
| Nivel de Virtualización | Nativo (Sin virtualizar) | Hardware y Software Completo  | Solo APIs / Capa Software |
| Fidelidad de Pruebas    | 100% (Prueba de oro)     | 90% - 95% (Muy Alta)          | 70% - 80% (Media)         |
| Consumo en Host (PC)    | Nulo (Usa su CPU/RAM)    | Muy Alto (2 - 4 GB RAM / CPU) | Bajo - Moderado           |
| Pruebas de Sensores/Red | Físicas y reales         | Simuladas por GUI / Telnet    | Limitadas / Mocks         |
| Integración CI/CD       | Compleja y costosa       | Excelente (Headless)          | Muy Rápida y Ligera       |
| Costo de Adquisición    | Alto (Múltiples equipos) | Gratuito (Con Android SDK)    | Gratuito e Integrado      |

## 2.2. APIs y Servicios Web para Aplicaciones Móviles

En las arquitecturas móviles modernas, los dispositivos cliente delegan el procesamiento intensivo y la persistencia de datos a servidores remotos mediante el consumo de APIs (Application Programming Interfaces) bajo el modelo arquitectónico REST.

### 2.2.1. Protocolo HTTP/HTTPS y Arquitectura REST

El protocolo HTTP (RFC 7231) sobre capas de transporte seguro TLS (HTTPS) define la semántica de comunicación cliente-servidor mediante métodos estándar: GET (consulta segura e idempotente), POST (creación de recursos), PUT (reemplazo completo), PATCH (modificación parcial) y DELETE (eliminación). Los códigos de estado categorizan las respuestas del servidor: 2xx (Éxito), 4xx (Errores del cliente como 400 Bad Request, 401 Unauthorized, 404 Not Found) y 5xx (Errores internos del servidor).

La arquitectura REST (Representational State Transfer) establece principios clave: comunicación sin estado (Statelessness), donde cada solicitud contiene toda la autenticación requerida; direccionamiento unívoco de recursos mediante URIs lógicas; e intercambio de cargas útiles estructuradas en formato JSON (JavaScript Object Notation).

### 2.2.2. Axios para Consumo de Servicios en Aplicaciones Móviles

¿Qué es? Axios es un cliente HTTP isomórfico basado en Promesas ECMAScript (async/await) compatible con Node.js, navegadores y entornos móviles multiplataforma como React Native y Capacitor.

¿Cómo funciona su Arquitectura? Axios encapsula las llamadas nativas de red utilizando XMLHttpRequest o adaptadores HTTP nativos. Integra transformación automática de datos JSON, soporte para cancelación de solicitudes mediante AbortController, control explícito de timeouts y una arquitectura de Interceptores de Red.

**Interceptores de Request y Response:** El interceptor de petición permite inyectar dinámicamente cabeceras de autorización (tokens Bearer JWT) y registrar marcas de tiempo antes de que la solicitud salga del dispositivo. El interceptor de respuesta captura las respuestas del servidor, calcula la latencia en milisegundos y normaliza de forma unificada los códigos de error HTTP para proteger la interfaz de usuario.

**Tabla 2. Comparativa Técnica: Cliente Axios vs Fetch API Nativa**

*Nota: Tabla comparativa estructurada bajo parámetros técnicos de ingeniería.*

| Característica                     | Cliente Axios                                         | Fetch API Nativa                                           |
|------------------------------------|-------------------------------------------------------|------------------------------------------------------------|
| <b>Transformación JSON</b>         | Automática (Serializa y parsea data)                  | Manual (Requiere invocar <code>response.json()</code> )    |
| <b>Interceptores Globales</b>      | Nativos (Request y Response)                          | No soportados (Requiere envoltorios custom)                |
| <b>Manejo de Errores (4xx/5xx)</b> | Rechaza la promesa automáticamente                    | Resuelve con éxito (Obliga a validar <code>res.ok</code> ) |
| <b>Control de Timeouts Móviles</b> | Nativo mediante propiedad <code>timeout: 10000</code> | Requiere <code>AbortSignal.timeout()</code> manual         |
| <b>Cancelación de Peticiones</b>   | Soporte con <code>AbortController</code>              | Soporte con <code>AbortController</code>                   |
| <b>Protección XSRF / Headers</b>   | Integrada por defecto                                 | Configuración manual                                       |

## 2.3. Proceso de Generación y Compilación de APK (Android Package)

El archivo APK es el formato ejecutable y paquete de distribución para el sistema operativo Android. Funciona como un contenedor comprimido (basado en formato ZIP) que agrupa todos los elementos necesarios para la instalación de la aplicación.

### 2.3.1. Anatomía Interna de un Archivo APK

- **AndroidManifest.xml:** Manifiesto binario con permisos (INTERNET), actividades, servicios y configuraciones de hardware.
- **classes.dex:** Bytecode ejecutable en formato Dalvik Executable generado por el compilador D8/R8 para el runtime ART.
- **resources.arsc:** Tabla de recursos precompilados que almacena identificadores numéricos, textos, dimensiones y temas.
- **res/ y assets/:** Recursos visuales compilados (layouts, mipmaps) y archivos estáticos (fuentes, bundles JavaScript).
- **lib/:** Bibliotecas nativas compiladas en C/C++ (.so) segmentadas por arquitectura ABI (arm64-v8a, armeabi-v7a, x86\_64).
- **META-INF/:** Certificados criptográficos y manifiestos de firma digital (CERT.RSA, CERT.SF, MANIFEST.MF).

### 2.3.2. Pipeline de Compilación, Debug vs. Release y Firmado Digital

El pipeline de compilación de Android transforma el código fuente mediante las siguientes etapas: 1) Compilación de código a bytecode Java (.class) y posterior transformación a bytecode Dalvik (classes.dex) mediante el compilador D8/R8 con ofuscación y reducción de código; 2) Compilación de recursos mediante AAPT2 (Android Asset Packaging Tool); 3) Ensamblado del paquete APK por Gradle; 4) Alineación de memoria con zipalign; 5) Firma digital criptográfica con apksigner.

**Variantes Debug vs Release:** La variante Debug incluye banderas de depuración (`android:debuggable="true"`) y se firma con un certificado genérico (`debug.keystore`). La variante Release optimiza el código mediante ofuscación y Tree Shaking con R8/ProGuard, y exige la firma con un Keystore privado protegido por contraseña.

**Zipalign y Apksigner:** Zipalign alinea los archivos no comprimidos del ZIP en múltiplos de 4 bytes (32 bits), permitiendo al kernel de Android leer recursos directamente mediante mapeo de memoria `mmap()` sin consumos innecesarios de RAM. Apksigner aplica firmas criptográficas en esquemas v1 (JAR), v2 (bloque completo del APK) y v3 (rotación de claves), garantizando autenticidad e integridad contra manipulaciones de código.

## 3. DESARROLLO DEL EJEMPLO PRÁCTICO

### 3.1. Arquitectura de la Aplicación Móvil Desarrollada

Como demostración práctica se construyó la aplicación móvil "MobileStore & API Hub", diseñada con una arquitectura por capas desacoplada: Capa de Presentación (Interfaz responsiva con viewport de smartphone), Capa de Controladores (src/app.js), Capa de Servicios REST (src/api/productService.js) y Capa de Cliente HTTP (src/api/apiClient.js).

### 3.2. Implementación del Cliente Axios con Interceptores

El módulo apiClient.js centraliza la comunicación con la API REST remota, configurando un timeout de 10 segundos para tolerancia a redes móviles, inyectando cabeceras de autorización Bearer JWT y calculando la latencia en milisegundos:

```
// src/api/apiClient.js - Instancia Centralizada con Interceptores
const axios = require('axios');

const apiClient = axios.create({
  baseURL: 'https://fakestoreapi.com',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'X-Client-Platform': 'Android-Mobile-App'
  }
});

// 1. Interceptor de Petición (Request): Inyección de Token y Telemetría
apiClient.interceptors.request.use((config) => {
  config.metadata = { startTime: new Date() };
  config.headers['Authorization'] = 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
  console.log(`[HTTP OUT] [${config.method.toUpperCase()}] -> ${config.url}`);
  return config;
});

// 2. Interceptor de Respuesta (Response): Latencia y Mapeo de Errores
apiClient.interceptors.response.use(
  (response) => {
    const duration = new Date() - response.config.metadata.startTime;
    return { success: true, status: response.status, data: response.data, durationMs: duration };
  },
  (error) => {
    const status = error.response ? error.response.status : 'ERR_NETWORK';
    return Promise.reject({ success: false, status, message: error.message });
  }
);

module.exports = apiClient;
```

### 3.3. Resultados de Pruebas de Integración y Consumo REST

Se ejecutó la suite automatizada test-axios-services.js contra la API REST pública, validando satisfactoriamente las operaciones CRUD con medición de latencia:

```
=====
PRUEBAS DE INTEGRACIÓN DE SERVICIOS REST CON AXIOS
Tutor: Ing. Luis Calo | Entorno: Node.js / Runtime Móvil
=====
>>> [TEST 1] GET /products?limit=3    -> 200 OK | Latencia: 3499ms | 3 items
>>> [TEST 2] GET /categories          -> 200 OK | Latencia: 196ms | 4 categorías
>>> [TEST 3] POST /products           -> 201 Created | Latencia: 198ms | ID: 21
>>> [TEST 4] PUT /products/1          -> 200 OK | Latencia: 197ms | Modificado
>>> [TEST 5] DELETE /products/1       -> 200 OK | Latencia: 199ms | Confirmado
=====
RESULTADO: TODAS LAS PRUEBAS DE CONSUMO REST FUERON EXITOSAS
=====
```

### 3.4. Pipeline de Construcción, Alineación y Firmado de APK

A través de los scripts build-tools/generate-keystore.sh y build-tools/build-apk-workflow.sh se ejecutó la secuencia de compilación:

```
# 1. Generación de Keystore con algoritmo RSA de 2048 bits
keytool -genkey -v -keystore release-key.jks -alias mobileappkey -keyalg RSA -keysize 2048 -validity 10000

# 2. Alineación de límites de 4 bytes con zipalign (Optimización de memoria mmap)
zipalign -p -f -v 4 ./dist-apk/app-unsigned-unaligned.apk ./dist-apk/app-unaligned.apk

# 3. Firmado criptográfico con apksigner (Esquemas v1, v2 y v3)
apksigner sign --ks release-key.jks --out ./dist-apk/MobileStore-Release-Signed.apk ./dist-apk/app-unaligned.apk

# 4. Verificación de firma digital
apksigner verify --verbose ./dist-apk/MobileStore-Release-Signed.apk
```

## 4. CONCLUSIONES

1. Complementariedad Estratégica de Entornos de Prueba: La investigación demostró que los emuladores basados en AVD/QEMU y los simuladores de software son herramientas óptimas para acelerar el ciclo de desarrollo temprano e integración continua (CI/CD). No obstante, las pruebas en dispositivos físicos reales son indispensables e insustituibles para auditar métricas críticas como el consumo de batería (mAh), la disipación térmica de la CPU y la respuesta táctil nativa.
2. Superioridad Técnica de Axios en Aplicaciones Móviles: Frente a la API Fetch nativa, Axios proporciona una arquitectura más robusta y desacoplada gracias a su sistema de interceptores globales. Esto permite centralizar la inyección de credenciales JWT Bearer, auditar la latencia de red en tiempo real y estandarizar el tratamiento de códigos de error HTTP (4xx y 5xx) en una sola capa de software.
3. Resiliencia ante la Inestabilidad de Conectividad Móvil: El establecimiento de tiempos de espera explícitos (timeouts) y la captura semántica de excepciones en Axios protegen el hilo principal de la interfaz de usuario (UI Thread) ante interrupciones de conectividad 4G/5G, garantizando una experiencia de usuario sólida y predecible.
4. Eficiencia de Memoria y Seguridad Criptográfica en Android: El proceso de alineación a 4 bytes mediante zipalign optimiza sustancialmente el consumo de memoria RAM en el dispositivo mediante lectura directa mmap(). Complementariamente, el firmado con apksigner bajo esquemas v2 y v3 asegura la integridad inmutable del archivo APK, evitando alteraciones maliciosas del código.
5. Validez del Enfoque por Capas en el Desarrollo de Software: La arquitectura modular implementada en el proyecto "MobileStore & API Hub" demostró que una separación estricta de responsabilidades entre la capa de presentación, los servicios de negocio y el cliente HTTP facilita el mantenimiento, simplifica la ejecución de pruebas unitarias y optimiza el empaquetado final de la aplicación.

## 5. BIBLIOGRAFÍA

- Android Open Source Project. (2024). Android Debug Bridge (adb) architecture and command reference. Android Developers. <https://developer.android.com/tools/adb>
- Axios Community. (2024). Axios HTTP client documentation: Interceptors, instances and error handling. Axios HTTP. <https://axios-http.com/docs/intro>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).
- Google Developers. (2024a). Run apps on the Android Emulator and manage Android Virtual Devices (AVD). Android Studio Documentation. <https://developer.android.com/studio/run/emulator>
- Google Developers. (2024b). Sign your app: APK Signature Scheme v2, v3 and keytool management. Android Developers Guide. <https://developer.android.com/studio/publish/app-signing>
- Google Developers. (2024c). Reduce app size and optimize alignment with zipalign. Android Studio Tools. <https://developer.android.com/tools/zipalign>
- IETF (Internet Engineering Task Force). (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231). <https://datatracker.ietf.org/doc/html/rfc7231>
- Martin, R. C. (2018). Clean Architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Meier, R., & Lake, I. (2018). Professional Android (4th ed.). Wrox - John Wiley & Sons.
- Smyth, N. (2023). Android Studio Panda Essentials - Kotlin Edition: Developing Android Apps Using Android Studio and Kotlin. Payload Media.